DNN Programming Assignment 2

Viradiya Tushar Rajeshbhai Student ID: 2026aj05073

Problem 1: data, verification and HAR classification

UCI HAR uses nine channels in the prescribed order: body acceleration xyz, body gyroscope xyz, total acceleration xyz. Shapes are $7352 \times 128 \times 9$ (training source) and $2947 \times 128 \times 9$ (test), with labels remapped 1–6 to 0–5. The 21 training and 9 test subjects are disjoint. For each seed, NumPy's prescribed choice holds out four training subjects; normalization uses that seed's training portion only. Figure 1 shows class balance; constant channels are rejected.

The variants table is absent: $h = 64$ and seeds 0/1/2 are provisional. All recurrent classifiers use the last hidden state, one library layer, no dropout, Adam 10^{-3}, batch 128, 10 epochs and norm-1 clipping. Best checkpoints use validation macro-F1 (earliest tie). All reported spread is sample std over three seeds; timing is training-only CPU, four threads, not T4. Test data never select checkpoints.

Scratch cells use registered matrices, tanh RNN, LSTM i/f/g/o gates and GRU r/z/n gates, including PyTorch's reset on the recurrent candidate bias. Eight forward cases per cell cover two dtypes, two seeds and nonzero states. Maximum errors are $1.19 \cdot 10^{-7}$ (RNN), $2.38 \cdot 10^{-7}$ (LSTM/GRU), below 10^{-5}; backward errors are at most $2.22 \cdot 10^{-16}$. For $g = 1, 4, 3$ gates, single-bias count is $gh(d+h+1)$; PyTorch/custom count is $gh(d+h+2)$. At $d = 9, h = 64$, these are 4736/4800, 18944/19200 and 14208/14400. Classifier heads add $6h + 6$ parameters.

Table 1. HAR recurrent comparison

Model	Accuracy (%)	Macro-F1	Parameters	sec/epoch
RNN	70.52 ± 4.91	0.6814 ± 0.0495	5190	0.677
LSTM	87.71 ± 0.29	0.8768 ± 0.0023	19590	1.182
GRU	88.19 ± 1.04	0.8819 ± 0.0104	14790	2.279

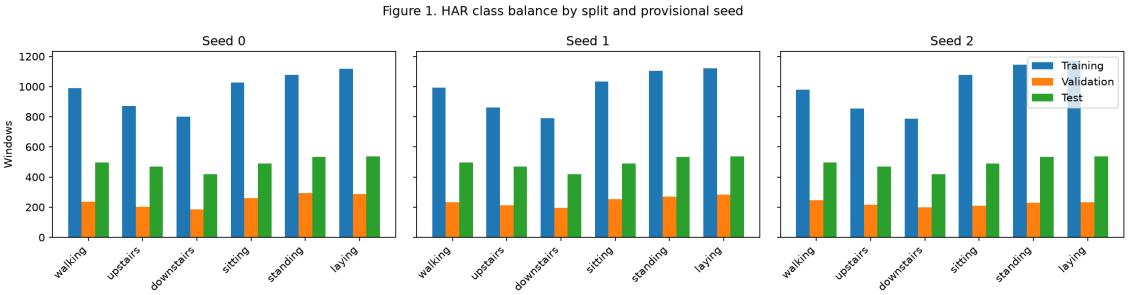


Figure 1. Class balance for all three subject splits and the test set.

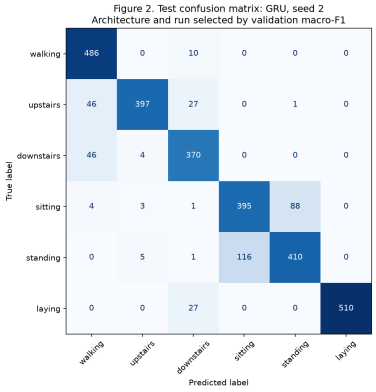


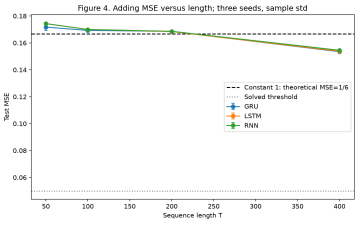
Figure 2. Validation-selected GRU, seed 2, epoch 9. Sitting/standing account for 204 cross-errors (88+116). Mean validation macro-F1 selects GRU (0.9104), then seed 2; it is not the highest-test-score run. Table 1 shows the strong classification gain from gating. The custom cells are checked against library layers; library recurrent layers are used only in permitted experiments.

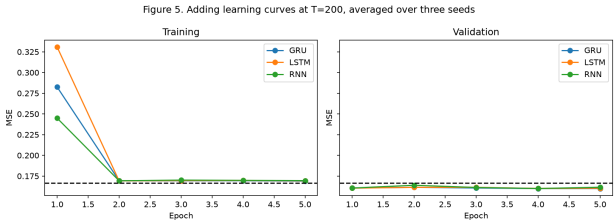
Problem 1: adding problem, width and individual ablations

The supplied generator is unchanged: two markers, one per half; targets equal their marked-value sum. All $T \in \{50, 100, 200, 400\}$ use 10000/1000/1000 train/validation/test examples with data seeds 0/1/2. Models use $h = 64$, batch 128, Adam 10^{-3}, clipping 1 and five epochs; lowest validation MSE selects a checkpoint. Model seeds are 0/1/2. Solved means test MSE < 0.05. Table 2 and Figures 4–5 show no solved run (0/3 in every model/length group). The short budget was fixed before testing.

Table 2. Adding test MSE; every group solved 0/3

T	RNN	LSTM	GRU	Constant 1
50	0.1743 ± 0.0005	0.1743 ± 0.0003	0.1717 ± 0.0025	0.1754
100	0.1700 ± 0.0005	0.1700 ± 0.0008	0.1693 ± 0.0008	0.1697
200	0.1686 ± 0.0008	0.1684 ± 0.0005	0.1685 ± 0.0003	0.1679
400	0.1545 ± 0.0003	0.1538 ± 0.0005	0.1535 ± 0.0003	0.1527





Figures 4–5. Test MSE versus length (left); T=200 learning curves (right).

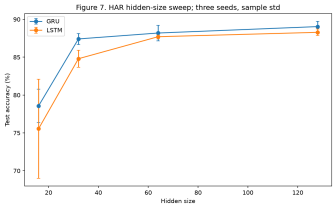
Table 3/Figure 7 retain the same 10-epoch HAR protocol; six $h = 64$ runs are reused. ID ends in 3 (odd): Table 4/Figure 8 compare initial LSTM effective forget bias 0/1 at T=200, five epochs, three paired seeds; the other weights and shuffle are identical.

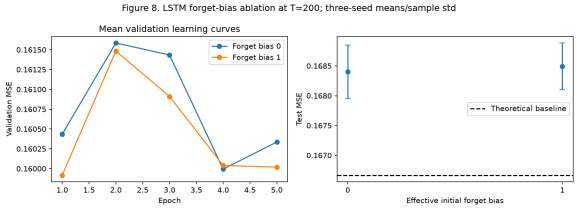
Table 3. HAR sweep: LSTM (left), GRU (right)

h	LSTM acc. (%)	Params	sec	GRU acc. (%)	Params	sec
16	75.55 ± 6.53	1830	0.30	78.57 ± 2.23	1398	1.12
32	84.80 ± 1.12	5702	0.54	87.41 ± 0.73	4326	1.59
64	87.71 ± 0.29	19590	1.18	88.19 ± 1.04	14790	2.28
128	88.28 ± 0.43	71942	3.00	89.04 ± 0.70	54150	4.18

Table 4. Initial forget-bias ablation

Bias	Test MSE	Solved
0	0.16840 ± 0.00045	0/3
1	0.16849 ± 0.00039	0/3





Figures 7–8. Width trade-offs (left); initial forget-bias comparison (right).

Problem 1: gradient evidence and one-page analysis

Figure 6. T=100 hidden gradients before/after training; three-seed means

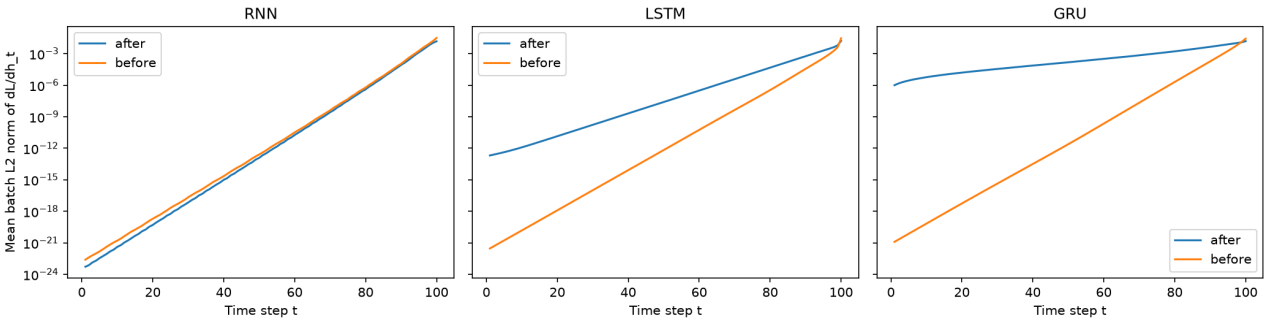


Figure 6. At T=100, mean batch L2 norms of actual $\partial L/\partial h_t$; log scale, before/after training, three-seed means. A fixed independent 32-example probe uses seed 3. Float64 custom unrolls reproduce library predictions and all recurrent parameter gradients. LSTM retains a separate cell-state graph.

Figure 6 shows severe attenuation of hidden-state gradients toward early time steps at T=100. After training, the ratio of the mean norm at t=1 to that at t=100 is about 3.6×10^{-22} for RNN, 1.2×10^{-11} for LSTM and 6.5×10^{-5} for GRU. The gated models retain a stronger early hidden-state signal in these probes, especially GRU. Predictions and recurrent parameter gradients match the library models, checking that the unrolled computation graph is correct. These norms also depend on the loss and state trajectory. LSTM's separate cell-state path is not fully represented by dL/dh_t, so the plot cannot establish a complete memory ranking.

Table 2 and Figures 4–5 show that none of the models solved the adding task at any tested length under five epochs: all 36 runs exceed MSE 0.05 and remain near their empirical constant-1 baselines. Training at T=200 plateaus around 0.17. The lower MSE at T=400 also occurs for constant prediction and is not evidence of improved memory. The short budget prevents the anticipated contrast between successful gated models and a failing RNN; stronger gradient signals alone did not produce successful marked-value addition here.

The odd-ID experiment isolates initial forget bias while pairing other weights and shuffle sequences. Table 4/Figure 8 give test MSE 0.16840±0.00045 for bias 0 and 0.16849±0.00039 for bias 1, with zero solved runs for either. This tiny difference relative to seed variation does not support a reliable benefit or harm from bias 1 under this budget. It does not rule out a benefit after longer training.

On HAR, Table 1 shows RNN accuracy 70.52±4.91%, LSTM 87.71±0.29% and GRU 88.19±1.04%. Figure 2 highlights sitting/standing confusion in the validation-selected GRU. Gated cells improve this classification task despite the unsuccessful adding experiment. Table 3/Figure 7 show diminishing accuracy gains with width: GRU grows from 87.41±0.73% at h=32 to 89.04±0.70% at h=128, while parameters grow from 4,326 to 54,150. LSTM h=128 reaches 88.28±0.43% with 71,942 parameters. CPU training times favor LSTM over comparable GRU widths on this backend; they do not measure deployment inference energy.

For a memory-constrained wearable, I would prototype GRU h=32. It uses about 71% fewer parameters than GRU h=64 for a 0.78 percentage-point lower mean accuracy, and beats LSTM h=32 in this sweep. Its float32 weights occupy about 16.9 KiB, excluding activations and runtime overhead. This recommendation follows the measured accuracy/size trade-off; battery life and latency still require device measurements. Three seeds, provisional variant settings and short training budgets limit generalization and statistical certainty.

Problem 2: scratch Transformer and HAR encoder

Custom scaled dot-product attention computes

$$A = \mathrm{softmax}(QK^\top/\sqrt{d_k} + M), \qquad O = AV,$$

where forbidden logits are $-\infty$. Boolean True means visible. Every query needs at least one visible key; fully masked rows are rejected. Row sums, future zeros, shapes and random output equivalence pass. Maximum SDPA reference error is $3.58 \cdot 10^{-7}$ and input-gradient error $8.88 \cdot 10^{-16}$. Only tests use framework attention/Transformer implementations.

For the **illustrative, not instructor-supplied** three-token check, $Q = K = 0$ with width 2 and $V = [(1,0),(0,2),(3,4)]$. Unmasked rows have weights $(1/3, 1/3, 1/3)$ and output $(4/3, 2)$. Causal rows have weights $(1,0,0), (1/2,1/2,0), (1/3,1/3,1/3)$ and outputs $(1,0), (1/2,1), (4/3,2)$, verified by code. The instructor's specified fixture is absent; the paper [1] gives the equation, not this numerical example.

Multi-head attention has separate biased Q/K/V/O projections and head reshaping. The block uses pre-LayerNorm (eps= 10^{-5}), residual branches, ReLU FFN width $4d$, and dropout 0.1 on activated FFN and branch outputs. Tests turn dropout off: MHA and block errors are $1.80{\cdot}10^{-16}$ and $8.88{\cdot}10^{-16}$. Sinusoidal $PE_{p,2i} = \sin(p/10000^{2i/d})$, $PE_{p,2i+1} = \cos(p/10000^{2i/d})$ matches scalar calculations within $5.96 \cdot 10^{-8}$; causal perturbation and parameter-gradient checks pass.

Block matrix count is $4d^2{+}2d\,d_{ff} = 12d^2$. Attention biases add $4d$, FFN biases $5d$, and two LayerNorms $4d$: total $12d^2 + 13d$. At $d = 128$ this is 196608+1664=198272, matching actual parameters. At $d = 64$ it is 49984. Two blocks, patch embedding, CLS, final norm and 6-class head give 105222; learned positions add 1088. The untied 65-character language model has 108481 parameters, including embeddings/head.

Task A uses 8-step patches (72 values), a $d = 64$ embedding, CLS, two blocks/four heads, then normalized CLS classification. Nine runs compare none/sinusoidal/learned positions, using the exact P1 subjects, frozen training statistics, batch 128, Adam10^{-3}, clipping 1 and 10 epochs. Architecture, dropout and patching differ from recurrent models.

Table 5. HAR positional comparison, three seeds

Positions/model	Accuracy (%)	Macro-F1	Params	sec/epoch
learned	91.20 ± 0.53	0.9121 ± 0.0057	106310	1.521
none	90.54 ± 1.82	0.9045 ± 0.0182	105222	1.419
sinusoidal	90.61 ± 1.02	0.9062 ± 0.0106	105222	1.544
P1 GRU	88.19 ± 1.04	0.8819 ± 0.0104	14790	2.279

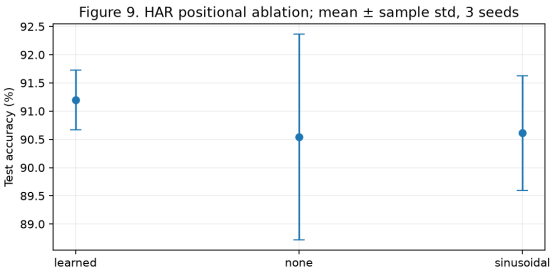


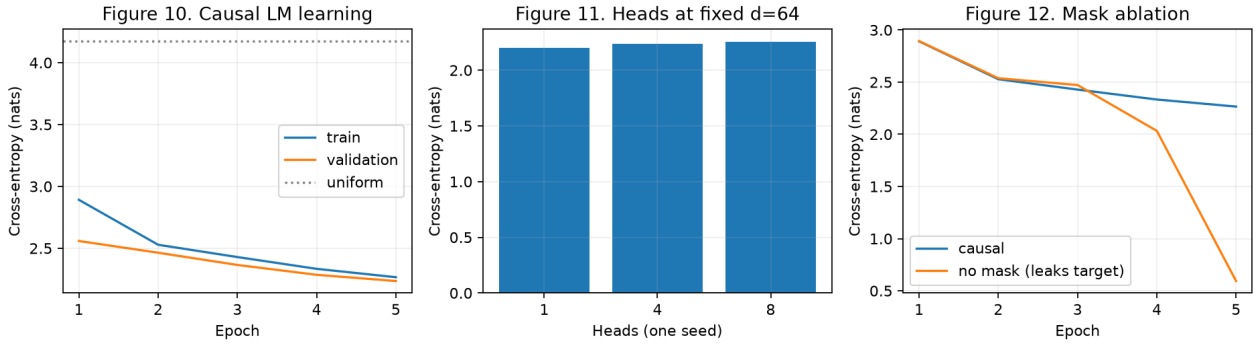
Figure 9. HAR accuracy with sample std. Mean validation F1 chooses sinusoidal (0.9276), not learned (0.9144), despite learned's higher test mean. The selected encoder improves accuracy by 2.42 points over GRU, using 7.11× the parameters. Patching reduces 128 steps to 17 tokens including CLS; backend time is not wearable energy.

Problem 2: causal language modeling, heads and masking

Tiny Shakespeare has 1115394 characters and 65 distinct characters. The whole-text vocabulary is sorted; first 90% (1003854) train, last 10% (111540) validation. Contiguous nonoverlapping 128-step next-character blocks are not shuffled; fewer than 128 trailing transitions per split are omitted. No targets cross the split boundary. CPU uses the assignment FAQ exception: $d = 64$, two layers (rather than $d = 128$/four), context 128/four heads. All comparisons use five full epochs, batch 64, seed 0, Adam10^{-3}, dropout 0.1 and norm1 clipping. Minimum validation loss selects checkpoints. Head/mask variants start with identical parameter values; all have 108481 parameters.

Table 6. LM validation; BPC = nats / ln 2

Heads	Mask	Nats	BPC	sec/epoch
4	causal	2.23495	3.22434	22.09
1	causal	2.20279	3.17795	16.02
8	causal	2.25598	3.25470	32.62
4	none: leakage	0.31769	0.45834	25.00



Figures 10–12. Learning curve (left), fixed-width head ablation (middle), and mask-removal training loss (right). Uniform baseline is $\ln 65 = 4.17439$ nats. Main validation loss falls 2.55856 to 2.23495 at epoch 5 and is still improving. The smaller CPU model/short budget plausibly explain weak results; neither effect is isolated. The one-head run wins here, but one seed cannot establish a general ranking. More heads retain the same matrices while storing more $T \times T$ softmax maps.

Without the mask, token t+1 exposes the target at t. Final training loss collapses to 0.59362 versus causal 2.26579. Apparent validation 0.31769 nats/0.45834 BPC is **contaminated**, not autoregressive quality. Changing future tokens leaves the trained causal prefix logits unchanged (max 0), but changes unmasked logits by 7.15354. At generation the missing future targets expose the failure:98.4% of its 500 characters are newlines and only 4 characters are distinct (temperature 0.8).

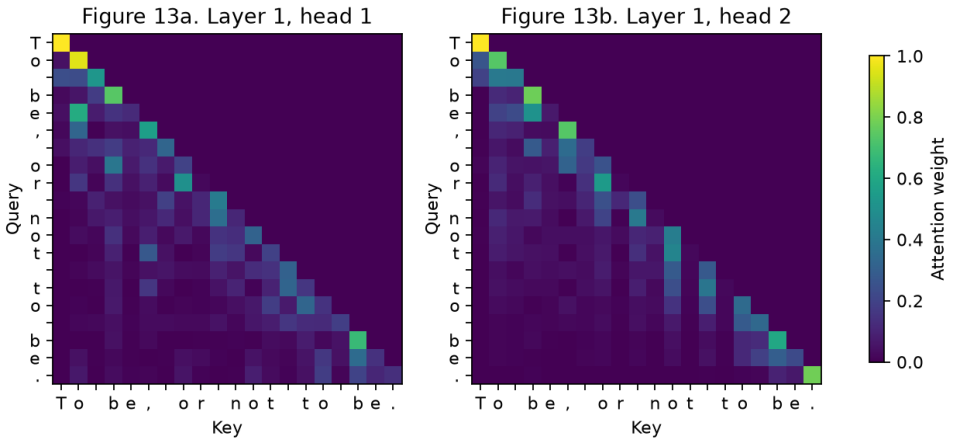
Three labelled 500-new-character continuations are saved in generated_samples.txt: prompt `ROMEO:\n`, top-k 20, seed 700, temperatures 0.2/0.8/1.5. Table 7 describes these draws, not predictive model entropy or repeated-sample significance.

Table 7. Main-model sample diversity

Temperature	Distinct chars	Empirical entropy (bits)
0.2	14	2.920
0.8	31	4.167
1.5	44	4.688

At 0.2, repeated high-probability `the`/`and` fragments dominate. At 0.8, wordlike variation and speaker-like line breaks increase, but spelling remains weak. At 1.5, case/punctuation become more varied and coherence declines. All remain weak samples; no fluent Shakespeare claim is supported.

Problem 2: attention interpretation and context scaling

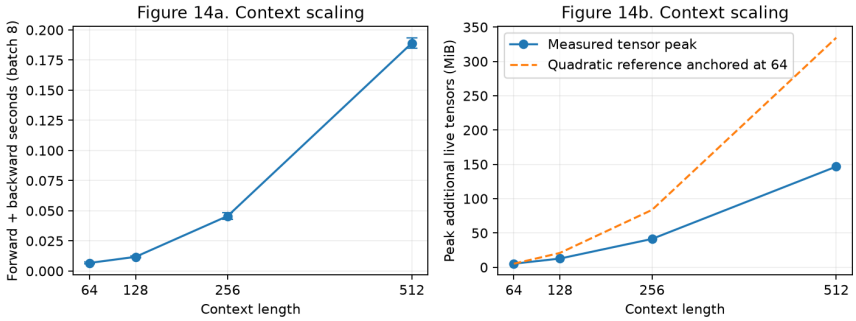


Figures 13a–b. Trained layer 1, heads 1/2 on "To be, or not to be." Rows are queries and columns keys. Both favor current/nearby characters with different off-diagonal patterns. Mean diagonal mass 0.3616/0.3736; first-token mass 0.0721/0.0747. Future entries are exactly zero; first row has only one visible key. One short character sentence does not prove syntactic specialization or causal explanation.

Measure the same causal model at batch 8, float32, eval/dropout off, contexts 64/128/256/512. Two warmups precede five timed forward+backward repeats. CPU memory reconstructs peak additional live tensor bytes from raw profiler allocation/free events, relative to existing model/input baseline; it excludes interpreter/model weights and allocator reservation. It is neither process RSS nor reserved GPU memory.

Table 8. Full-model context cost on CPU

Context	Time (ms), mean ± std	Peak tensor MiB	Score elements
64	6.684 ± 0.623	5.228	131072
128	11.736 ± 0.768	12.714	524288
256	45.613 ± 2.581	41.331	2097152
512	189.119 ± 4.451	146.753	8388608



Figures 14a–b. Time and incremental tensor peak versus context; dashed line is an anchored quadratic memory reference. With 8× context, score elements increase 64×, measured time 28.29×, tensor peak 28.07×. Attention products cost $O(T^2d)$; embeddings/FFN add linear-T work, so full-model ratios need not equal 64. Short-length timing includes overhead and cache effects. Long-context quality and T4/device latency are not evaluated.

All assertions pass; full notebooks execute top to bottom with outputs. Independent checkpoint audits reproduce HAR predictions, adding errors, LM losses, samples, gradients and selection. The 82-row ledger preserves 69 P1 and 13 P2 runs. The instructor hand fixture and variants table remain missing; the illustrative example and provisional settings are disclosed. Optional bonus is not attempted. Method and evidence, rather than reaching approximate sanity ranges, determine these conclusions.

AI-use statement and references (excluded from six-page limit)

OpenAI Codex was used for assignment review, project setup, data pipelines, tests, custom RNN, LSTM, GRU and Transformer implementations, experiment pipelines, debugging, execution, plots, result audits, analysis drafting, report generation and submission packaging. This includes AI-written from-scratch code. The user reported on 9 November 2026 that the professor permits full AI assistance, superseding the narrower AI restriction in the supplied assignment. This is reported authorization, not independent verification or a claim that the original document changed. The student is responsible for reviewing the work and confirming that authorization.

No pretrained weights or external datasets were used. Reference framework layers occur only in verification for restricted components; permitted library recurrent layers run P1 experiments. Experimental values are measured; failed adding runs and weak language samples are reported without fabricated success. Subject-specific variants remain provisional because the table is absent. The user-proposed paper [1] does not supply the missing worked numerical instructor fixture found in the brief.

[1] A. Vaswani et al. *Attention Is All You Need.*2017. Sections 3.2.1,3.2.2,3.5 and Table 1: attention, head projections, positions and complexity. https://arxiv.org/abs/1706.03762. The assignment's pre-LN requirement is followed; the original paper uses post-LN.

[2] UCI Machine Learning Repository. *Human Activity Recognition Using Smartphones.* Official nine-channel sensor windows and subject IDs. https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones.

[3] A. Karpathy. *char-rnn: Tiny Shakespeare input.txt.* https://github.com/karpathy/char-rnn/tree/master/data/tinyshakespeare.

[4] PyTorch reference documentation and installed class documentation: RNN, LSTM, GRU, functional scaled_dot_product_attention, MultiheadAttention and TransformerEncoderLayer (verification only). https://pytorch.org/docs/stable/nn.html.

[5] Course assignment: *DNN Programming Assignment–2: RNN, LSTM, GRU and Transformer*, supplied DOCX. Source of dataset/split/generator instructions, marking criteria and CPU exception. Portal link, variants table, instructor hand fixture and final deadline confirmation were not supplied.

Reproducibility: Python 3.14.4, PyTorch 2.15.0+cpu and exact package pins in requirements.txt; four CPU threads. P2 training-only time 10.22min; P1 adding 25.27min; these exclude validation/setup and do not estimate T4 runtime. Checkpoints/raw data are intentionally excluded from the submission and regenerated by the notebooks.